

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration: 1 Hour

Total Marks: 30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I Roshitha - 192424400 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule.

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- i. Movement allowed: Up, Down, Left, Right
- ii. Cost of each move = 1
- iii. Cannot pass through obstacles
- iv. Use Manhattan Distance heuristic

Using the above constraints and the BFS Search Algorithm, show step-by-step node expansion and priority queue updates, and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- a) Analyze all the given constraints and explain how they affect the robot's decision-making process.
- b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
- c) Demonstrate how the robot applies AI concepts such as: Intelligent agents, Rationality, Environment type.
- d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)
----------	-----------	---------------------	----------------------	----------------------

	e			
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification

CONSTRAINT-BASED PROBLEM SOLVING

CSA17 - Assessment Tool 2 - Artificial Intelligence

Name: Roshitha

Reg No: 192424400

1. Backtracking Search with Forward Checking for Doctor-Shift Assignment

1.1 Problem Definition

Three doctors (D1, D2, D3) must each be assigned one of three shifts (Morning, Afternoon, Night) subject to: D1 cannot take Night, D3 cannot take Morning, every shift is used by exactly one doctor, and D2's shift must precede D3's shift in the order Morning < Afternoon < Night. This is a Constraint Satisfaction Problem (CSP).

1.2 Constraint Analysis

Unary constraints restrict individual domains before any assignment begins: $D1 \neq \text{Night}$ removes Night from D1's domain, and $D3 \neq \text{Morning}$ removes Morning from D3's domain. The all-different constraint over shifts is a global constraint enforced every time a doctor is assigned. The ordering constraint $D2 < D3$ is a binary constraint that only becomes fully checkable once both D2 and D3 have candidate values, but it is used here to proactively prune each other's domain via forward checking as soon as one of the pair is assigned.

1.3 Search Strategy: MRV + Forward Checking

Rather than assigning doctors in a fixed D1-D2-D3 order and validating after the fact, this solution selects the next doctor to assign using the Minimum-Remaining-Values heuristic (whichever unassigned doctor currently has the fewest legal shifts) and immediately prunes the domains of the remaining doctors after every assignment.

Trace: Initial pruned domains are $D1 = \{\text{Morning}, \text{Afternoon}\}$, $D2 = \{\text{Morning}, \text{Afternoon}, \text{Night}\}$, $D3 = \{\text{Afternoon}, \text{Night}\}$. D1 has the fewest options and is assigned first: D1=Morning is tried and accepted, which removes Morning from D2 and D3. D2 and D3 are now tied at 2 options each; D2 is assigned next: D2=Afternoon is tried and accepted, which removes Afternoon from D3 (uniqueness) and also removes any D3 value not after Afternoon (ordering), leaving $D3 = \{\text{Night}\}$ only. D3=Night is then forced and satisfies every remaining constraint.

1.4 Final Valid Schedule

D1 → Morning, D2 → Afternoon, D3 → Night. No dead ends were encountered, so this branch required zero backtracking steps despite the search process being interleaved with constraint propagation at each assignment.

Python Code

```
def forward_check(domains, assigned_doctor, assigned_shift):
    new_domains = {d: set(v) for d, v in domains.items()}
    for d in DOCTORS:
        if d != assigned_doctor:
            new_domains[d].discard(assigned_shift)
    if assigned_doctor == "D2":
        new_domains["D3"] = {s for s in new_domains["D3"]
                               if SHIFT_ORDER[s] > SHIFT_ORDER[assigned_shift]}
    if assigned_doctor == "D3":
```

```

        new_domains["D2"] = {s for s in new_domains["D2"]
                               if SHIFT_ORDER[s] < SHIFT_ORDER[assigned_shift]}
    return new_domains

def select_unassigned_variable(domains, assignment):
    unassigned = [d for d in DOCTORS if d not in assignment]
    return min(unassigned, key=lambda d: len(domains[d]))    # MRV heuristic

```

Output

Final Valid Schedule: {'D1': 'Morning', 'D2': 'Afternoon', 'D3': 'Night'}

2. BFS Search in a 5×5 Grid Environment

2.1 Problem Definition

A robot must travel from S=(0,0) to G=(4,4) on a 5×5 grid, moving only Up/Down/Left/Right at a cost of 1 per move, without crossing obstacle cells.

2.2 Problem Formulation

States: free grid cells. Initial state: S. Goal state: G. Actions: move into an in-bounds, non-obstacle neighbour. Path cost: number of moves. Heuristic: Manhattan distance $h(n)$ is computed and logged at every expansion for reference, although BFS's optimality guarantee here comes from the uniform move cost rather than from the heuristic.

2.3 Frontier / Queue Behaviour

The frontier is a FIFO queue seeded with S. On each iteration the front node is dequeued, printed together with its Manhattan distance to G and the remaining frontier contents, and then its unvisited non-obstacle neighbours are enqueued (checked in the order Right, Down, Left, Up) and marked visited immediately to avoid duplicate queueing. Expansion stops the moment G itself is dequeued.

2.4 Optimal Path and Cost

The search expands 17 nodes before reaching G. The reconstructed path is (0,0) → (0,1) → (0,2) → (1,2) → (2,2) → (2,3) → (2,4) → (3,4) → (4,4), an 8-move path, so the optimal cost is 8 — matching the Manhattan distance lower bound of 8 from the start node, which confirms optimality.

Python Code

```

def bfs_with_frontier_log(start, goal, grid):
    frontier = deque([start]); came_from = {start: None}; visited = {start}
    while frontier:
        current = frontier.popleft()
        print(f"Expand: node={current}, h(n)={manhattan(current, goal)}, "
              f"frontier_before={list(frontier)}")
        if current == goal:
            break
        for dx, dy in [(0,1),(1,0),(0,-1),(-1,0)]:
            nxt = (current[0]+dx, current[1]+dy)
            r, c = nxt
            if 0<=r<5 and 0<=c<5 and grid[r][c]==0 and nxt not in visited:
                visited.add(nxt); came_from[nxt] = current; frontier.append(nxt)
    # reconstruct path via came_from ...

```

Output

Optimal Path: [(0,0),(0,1),(0,2),(1,2),(2,2),(2,3),(2,4),(3,4),(4,4)]. Cost: 8.

3. Autonomous Rescue Robot — Online Uniform Cost Search

3.1 Constraint Analysis

The robot operates in a partially observable, initially unknown, dynamic grid where cells may be free, obstacles, or risky zones (extra cost of 2 on top of the base move cost of 1). Because sensing is limited to adjacent cells, the robot cannot compute a complete path before it starts moving; it can only plan against what it currently knows and must revise that plan as it senses new areas. Minimising total cost means the robot must sometimes accept a longer route in exchange for avoiding risky zones.

3.2 Solution Approach: Online UCS

Implementation steps: (1) maintain a knowledge map starting with only the current cell known, treating unsensed cells as optimistically free; (2) before each move, sense the four adjacent cells and update the knowledge map with any obstacles or risky zones discovered; (3) run Uniform Cost Search from the current cell to the goal using only the knowledge map, with risky cells costing $1+2=3$; (4) commit to only the first step of that plan, move there, and add its real cost to the running total; (5) repeat sensing and re-running UCS from the new position until the goal is reached.

3.3 Application of AI Concepts

Intelligent agent: perceives adjacent cells, maintains an internal knowledge map, and selects cost-minimising actions based on that model. Rationality: at every step it picks the action expected to minimise total cost given current knowledge, which is the correct standard for rationality under partial observability. Environment type: partially observable (only adjacent cells sensed), dynamic and initially unknown (the map is built up over time), and sequential (each move changes what becomes observable next).

3.4 Justification

Online UCS is appropriate because UCS natively handles the variable move costs introduced by risky zones — something BFS/DFS cannot optimise for — while the online sense-plan-move loop lets the robot operate without requiring the full map in advance, satisfying both the cost-minimisation and partial-observability requirements of the scenario.

Python Code

```
def online_ucs_rescue(start, goal, environment):
    known_map = {start: environment[start[0]][start[1]]}
    current, total_cost, travelled = start, 0, [start]
    while current != goal:
        known_map.update(sense_adjacent(current, environment))
        # UCS over known_map only, unexplored cells assumed free ...
        # take just the first step of the plan, then re-sense and re-plan
    return travelled, total_cost
```

Output

Least-Cost Path: [(0,0),(1,0),(2,0),(2,1),(2,2),(3,2),(3,3),(3,4),(4,4)]. Total Cost: 8.